nos4a2 (Ângelo Azevedo) Day 2 CTF Writeups

1. Forensics

a. Say Cheese

I should have done this chall faster.

The description was:

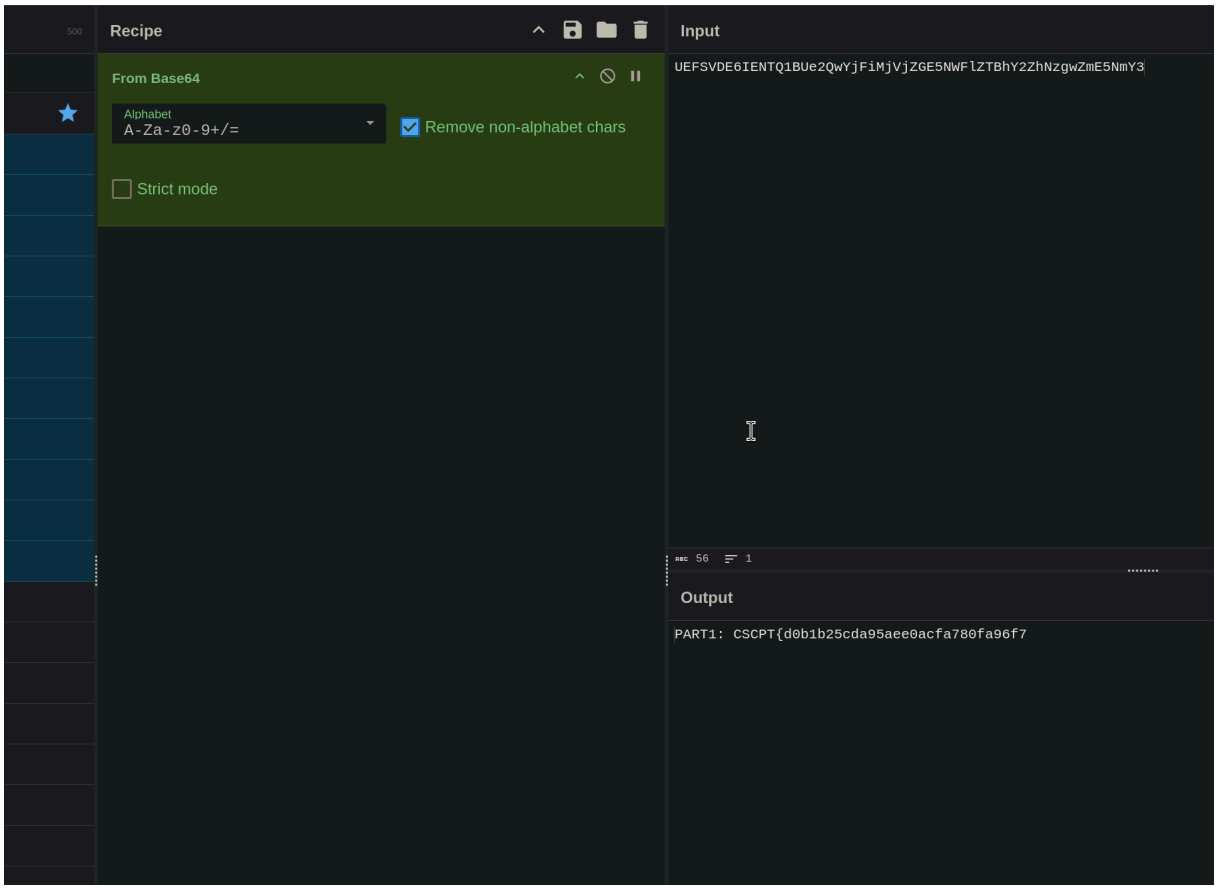
```
A suspect's phone photo was recovered. The visible image is useless, but the analyst
swears "the camera saw more than the screen." Find the flag.

The flag format is: CSCPT{}
```

The first thing I did was run exiftool on the given evidence.jpg.

```
forensics/say_cheese/7335ccfe3c048b6780a85106279d1437328c1c5a9d97c28fab8681ca8beea5c1
❯ exiftool evidence.jpg
ExifTool Version Number         : 13.59
File Name                       : evidence.jpg
Directory                       : .
File Size                       : 158 kB
File Modification Date/Time     : 2026:06:03 21:41:42+01:00
File Access Date/Time           : 0000:00:00 00:00:00
File Inode Change Date/Time     : 2026:07:19 09:08:29+01:00
File Permissions                : -rw-r--r--
File Type                       : JPEG
File Type Extension             : jpg
MIME Type                       : image/jpeg
JFIF Version                    : 1.01
Resolution Unit                 : None
X Resolution                    : 1
Y Resolution                    : 1
Exif Byte Order                 : Big-endian (Motorola, MM)
User Comment                    :
UEFSVDE6IENTQ1BUe2QwYjFiMjVjZGE5NWFlZTBhY2ZhNzgwZmE5NmY3
Thumbnail Offset                : 164
Thumbnail Length                : 3476
Image Width                     : 1536
Image Height                    : 1024
Encoding Process                : Baseline DCT, Huffman coding
Bits Per Sample                 : 8
Color Components                : 3
Y Cb Cr Sub Sampling            : YCbCr4:2:0 (2 2)
Image Size                      : 1536x1024
Megapixels                      : 1.6
Thumbnail Image                 : (Binary data 3476 bytes, use -b option to extract)
```

I got the User Comment and put it on CyberChef.



With that got the first part of the flag: `PART1: CSCPT{d0b1b25cda95aee0acfa780fa96f7`

After that I didn't look more to the exiftool (my biggest mistake).

I thought that the rest of the flag was some kind of LSB and so tried to search for it on https://aperisolve.fr.



Nothing.

Tried 20 different jpg LSB tools, I knew that zsteg didn't work for jpg files because of the compression that this files do, turning them lossy.

```
forensics/say_cheese/7335ccfe3c048b6780a85106279d1437328c1c5a9d97c28fab8681ca8beea5c1
❯ stegseek evidence.jpg ../../../rockyou.txt
StegSeek 0.6 - https://github.com/RickdeJager/StegSeek

[i] Progress: 99.44% (132.7 MB)
[!] error: Could not find a valid passphrase.

forensics/say_cheese/7335ccfe3c048b6780a85106279d1437328c1c5a9d97c28fab8681ca8beea5c1
❯ ./jsteg-linux-amd64 reveal evidence.jpg
jpeg does not contain hidden data
```

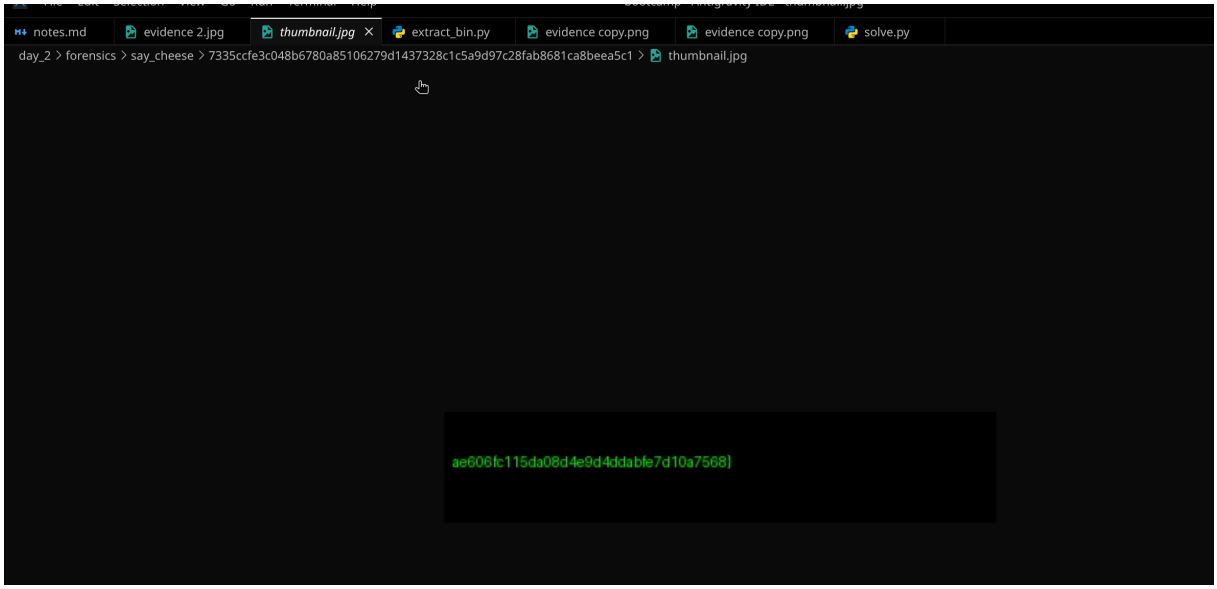
I even thought to create my own script to do it (glad I didn't lose time on that).

I then went back to Aperi'Solve and looked at everything again, noticed the Thumbnail Image on Exiftool.



And went on to extract it using:

```
forensics/say_cheese/7335ccfe3c048b6780a85106279d1437328c1c5a9d97c28fab8681ca8beea5c1
❯ exiftool -b -ThumbnailImage evidence.jpg > thumbnail.jpg
```



And with that got the second part of the flag finishing this challenge.

b. Liar's Archive

This chall description was:

```
Recovered from a suspect's drive: one image. It opens fine. It is also bigger than
any image of its dimensions has a right to be. The suspect scribbled a careless note
in the file's metadata. Find what they hid, and make it tell the truth.

Note: the hidden data inside the image has been obfuscated with a key. If one is
configured, it is exactly 4 bytes.

Note: intelligence confirms that the artifact where the flag resides has no
encryption. If it claims otherwise, it is not telling the truth.

The flag format is: CSCPT{}
```

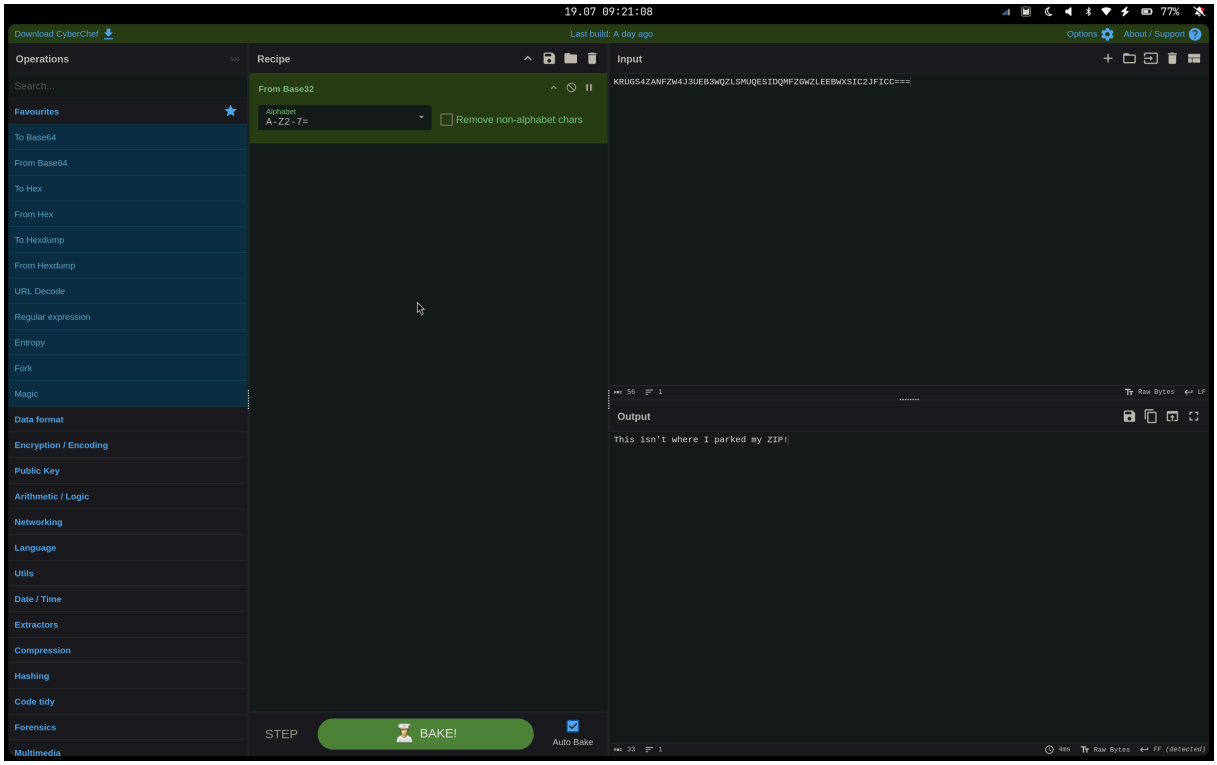
On this one I also started by running exiftool.

```
forensics/
liars_archive/9be2e46d554da8a7edccd09c7663a8931c982ea21872818ca24ac68d6d4e17b3
❯ exiftool evidence.png
ExifTool Version Number         : 13.59
File Name                       : evidence.png
Directory                       : .
File Size                       : 2.1 MB
File Modification Date/Time     : 2026:06:03 21:45:02+01:00
File Access Date/Time           : 0000:00:00 00:00:00
File Inode Change Date/Time     : 2026:07:19 09:18:32+01:00
File Permissions                : -rw-r--r--
File Type                       : PNG
File Type Extension             : png
MIME Type                       : image/png
Image Width                     : 1536
Image Height                    : 1024
```

```
Bit Depth                       : 8
Color Type                      : RGB
Compression                     : Deflate/Inflate
Filter                          : Adaptive
Interlace                       : Noninterlaced
Warning                         : [minor] Text/EXIF chunk(s) found after PNG IDAT
(may be ignored by some readers)
Comment                         :
KRUGS4ZANFZW4J3UEB3WQZLSMUQESIDQMFZGWZLEEBWXSIC2JFICC===
Image Size                      : 1536x1024
Megapixels                      : 1.6
```

Saw the command that I was already expecting by the chall description.

Went to CyberChef to decode it.



And got the message `This isn't where I parked my ZIP!`, showing that I should probably get a zip file somewhere.

I then ran binwalk and got nothing, also ran zsteg just to see if it found anything.

It showed:



That 182 bytes of extra data showed promising. On the description it showed the note:

```
Note: the hidden data inside the image has been obfuscated with a key. If one is
configured, it is exactly 4 bytes.
```

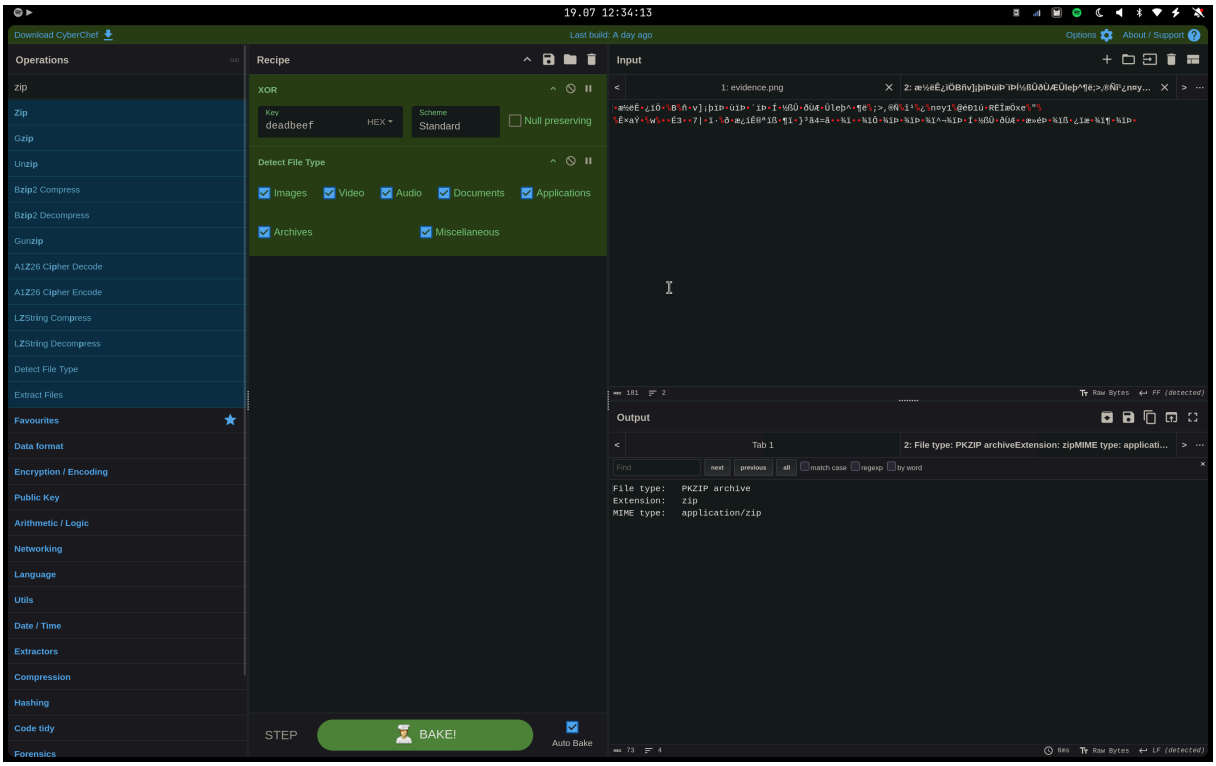
So the message was obfuscated with a key of 4 bytes.

Only when I used `xxd` I understood what the key could be:

```
forensics/
liars_archive/9be2e46d554da8a7edccd09c7663a8931c982ea21872818ca24ac68d6d4e17b3
❯ xxd evidence.png | tail -n 13
0020cd20: 8669 f200 0000 0049 454e 44ae 4260 828e  .i.....IEND.B`..
0020cd30: e6bd ebca adbf efd6 ad1f 421d f183 765d  ..........B...v]
0020cd40: a1fe efde adf9 efde adb4 efde adcd 8abd  ................
0020cd50: dfdb 9bf0 d9c6 9bdb 6c65 fe5e 9db6 eb1e  ........le.^....
0020cd60: 3b3e 2cae d119 eeb9 19bf 1d6e a479 3103  ;>,........n.y1.
0020cd70: 40e9 d031 fa9d 52cb cee6 d678 651d 2213  @..1..R....xe.".
0020cd80: 0c1e cad7 61dd 841b 7716 8e85 c933 8386  ....a...w....3..
0020cd90: 377c 9fef b701 f08e e6bf edca aeaa efdf  7|..............
0020cda0: adb6 ef7f 007d b3e3 343d e39e adbe ef99  .....}..4=......
0020cdb0: adbe efd4 adbe efde adbe efde adbe ef5e  ...............^
0020cdc0: acbe efde adcd 8abd dfdb 9bf0 d9c6 9b8e  ................
0020cdd0: e6bb e9de adbe efdf adbf efe6 adbe efb6  ................
0020cde0: adbe efde ad                             .....
```

I noticed the repeated `deadbeef` pattern, of exactly 4 bytes and tried that as a XOR key for the 182 extra bytes of data.

That gave me a PKZIP archive.



I tried to crack the password for a while, always getting the same error. (this output was not from a cracking tool, it was from a zip info tool but it was the same error)

```
forensics/
liars_archive/9be2e46d554da8a7edccd09c7663a8931c982ea21872818ca24ac68d6d4e17b3
❯ zipinfo download.zip
Archive:  download.zip
[download.zip]
  End-of-central-directory signature not found.  Either this file is not
  a zipfile, or it constitutes one disk of a multi-part archive.  In the
  latter case the central directory and zipfile comment will be found on
  the last disk(s) of this archive.
zipinfo:  cannot find zipfile directory in one of download.zip or
          download.zip.zip, and cannot find download.zip.ZIP, period.
```

Running `zipdetails` also gave an error:

```
forensics/
liars_archive/9be2e46d554da8a7edccd09c7663a8931c982ea21872818ca24ac68d6d4e17b3
❯ zipinfo download.zip
Archive:  download.zip
[download.zip]
  End-of-central-directory signature not found.  Either this file is not
  a zipfile, or it constitutes one disk of a multi-part archive.  In the
  latter case the central directory and zipfile comment will be found on
  the last disk(s) of this archive.
zipinfo:  cannot find zipfile directory in one of download.zip or
          download.zip.zip, and cannot find download.zip.ZIP, period.
```

I then when searching for the ZIP headers to see why the ZIP could be broken found some info about the `General Purpose Flag` with the `Bit 0` indicating `Encryption`.

```
001C Extra Length           0000 (0)
001E Filename               'secret.txt'
0028 PAYLOAD                .....0......p|..g.........W?.W#..cX9.......%..ck...a[
d.lX..p1...

0068 CENTRAL HEADER #1      02014B50 (33639248)
006C Created Zip Spec       14 (20) '2.0'
006D Created OS             03 (3) 'Unix'
006E Extract Zip Spec       14 (20) '2.0'
006F Extract OS             00 (0) 'MS-DOS'
0070 General Purpose Flag   0001 (1)
     [Bit  0]               1 'Encryption'
     [Bits 1-2]             0 'Normal Compression'
0072 Compression Method     0008 (8) 'Deflated'
#
# WARNING: Offset 0x78: 'Modification Time' value 0xC0DD0A1 (202231969)  decodes
 to '1986-18446744073709551615-13 26:05:02': not a valid DOS date/time
#
0074 Modification Time      0C0DD0A1 (202231969) 'Invalid Date or Time'
0078 CRC                    715D90EA (1901957354)
007C Compressed Size        76511373 (1985024883)
0080 Uncompressed Size      3B511373 (995169139)
0084 Filename Length        1373 (4979)
0086 Extra Length           3151 (12625)
0088 Comment Length         1373 (4979)
008A Disk Start             3151 (12625)
008C Int File Attributes    1373 (4979)
     [Bit 0]                1 'Text Data'
     [Bits 1-15]            0072 (114) 'Unknown'
008E Ext File Attributes    1372B151 (326283601)
     [Bit 0]                Read-Only
     [Bit 4]                Directory
     [Bit 6]                Device/Symbolic Link
     [Bit 8]                Temporary
     [Bit 12]               Offline
     [Bit 13]               Not Indexed
```

I then got stuck 1hr for the most stupidest reason possible. I correcly changed this bit from `1` to `0`:

```
forensics/
liars_archive/9be2e46d554da8a7edccd09c7663a8931c982ea21872818ca24ac68d6d4e17b3
❯ zipdetails --scan download.zip

(...)
0016 Uncompressed Size      00000047 (71)
001A Filename Length        000A (10)
001C Extra Length           0000 (0)
001E Filename               'secret.txt'
0028 PAYLOAD                .....0......p|..g.........W?.W#..cX9.......
%..ck...a[d.lX..p1...

0068 CENTRAL HEADER #1      02014B50 (33639248)
006C Created Zip Spec       14 (20) '2.0'
006D Created OS             03 (3) 'Unix'
006E Extract Zip Spec       14 (20) '2.0'
006F Extract OS             00 (0) 'MS-DOS'
0070 General Purpose Flag   0001 (1)
     [Bit  0]               1 'Encryption'
     [Bits 1-2]             0 'Normal Compression'
```

```
0072 Compression Method    0008 (8) 'Deflated'
#
# WARNING: Offset 0x78: 'Modification Time' value 0xC0DD0A1 (202231969)  decodes to
'1986-18446744073709551615-13 26:05:02': not a valid DOS date/time
#
0074 Modification Time     0C0DD0A1 (202231969) 'Invalid Date or Time'
0078 CRC                   715D90EA (1901957354)
007C Compressed Size       76511373 (1985024883)
0080 Uncompressed Size
```

I was changing the bit on the offset `0070`, when I change the on on `0006`:

```
forensics/
liars_archive/9be2e46d554da8a7edccd09c7663a8931c982ea21872818ca24ac68d6d4e17b3
❯ zipdetails --scan download\ \(Copy\).zip

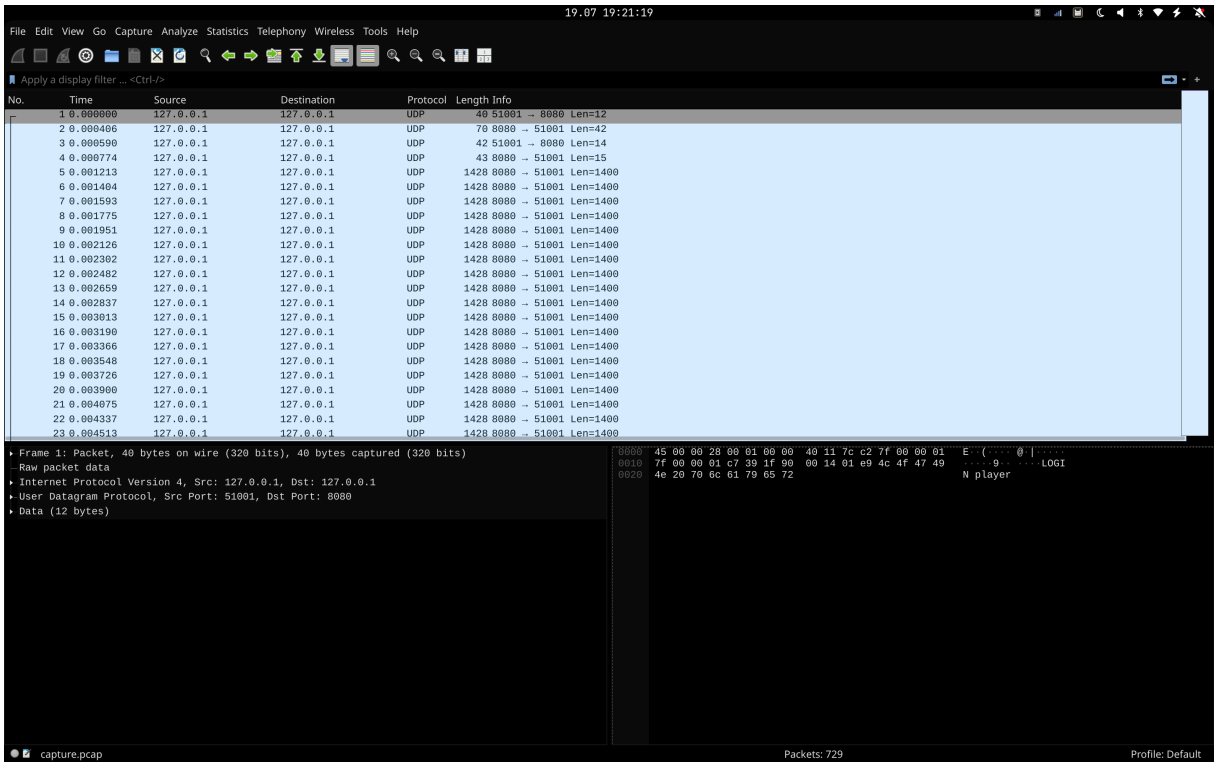
0000 LOCAL HEADER #1       04034B50 (67324752)
0004 Extract Zip Spec      14 (20) '2.0'
0005 Extract OS            00 (0) 'MS-DOS'
0006 General Purpose Flag  0000 (0)
     [Bits 1-2]            0 'Normal Compression'
0008 Compression Method    0008 (8) 'Deflated'
000A Modification Time     5CC3ADA1 (1556327841) 'Wed Jun  3 22:45:02 2026'
000E CRC                   0C83993D (209951037)
0012 Compressed Size       00000040 (64)
0016 Uncompressed Size     00000047 (71)
001A Filename Length       000A (10)
001C Extra Length          0000 (0)
001E Filename              'secret.txt'
0028 PAYLOAD               .....0......p|..g.........W?.W#..cX9.......
%..ck...a[d.lX..p1...
```

It got over the need for password and I was able to extract the ZIP getting to the `secret.txt` file containing the flag.

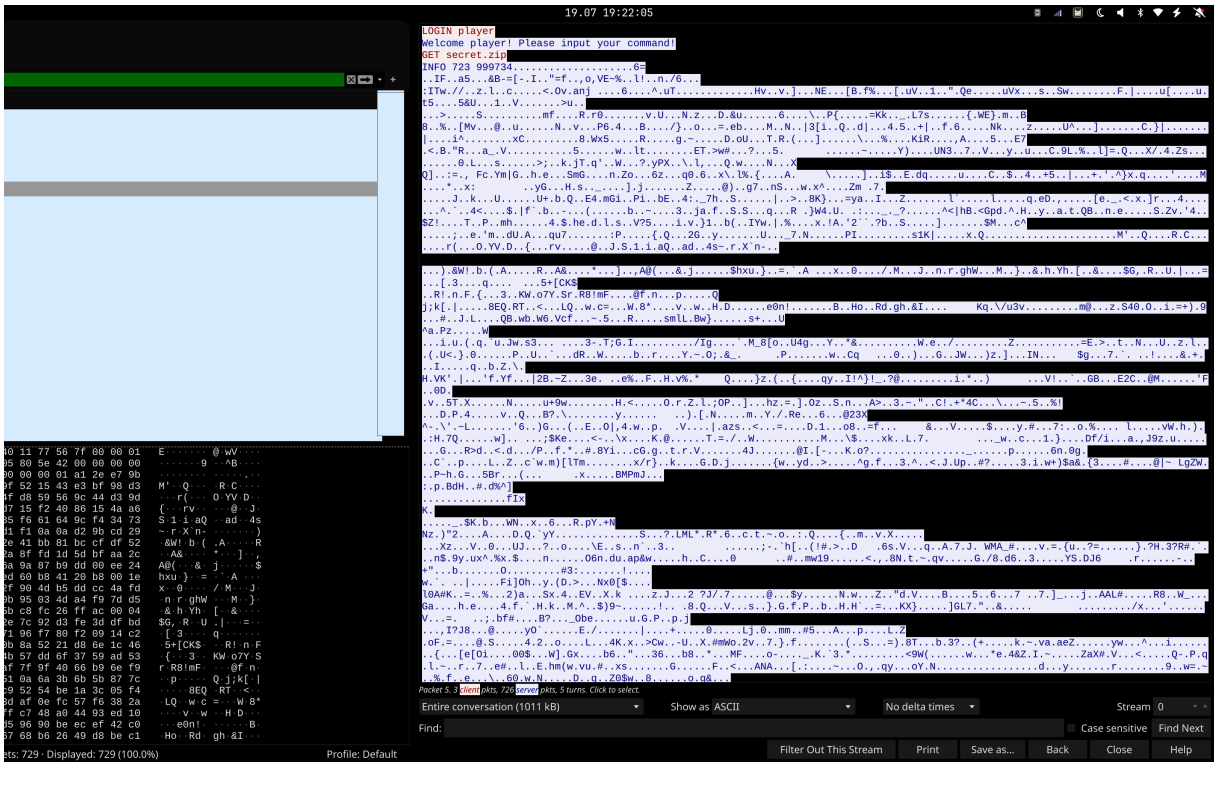
c. Out of Control

This chall was a good one but got me some headhaches.

We had a pcap file, I started by opening it and checking what it had inside.



So, a lot of UDP packets, pressed right click and followed the stream.



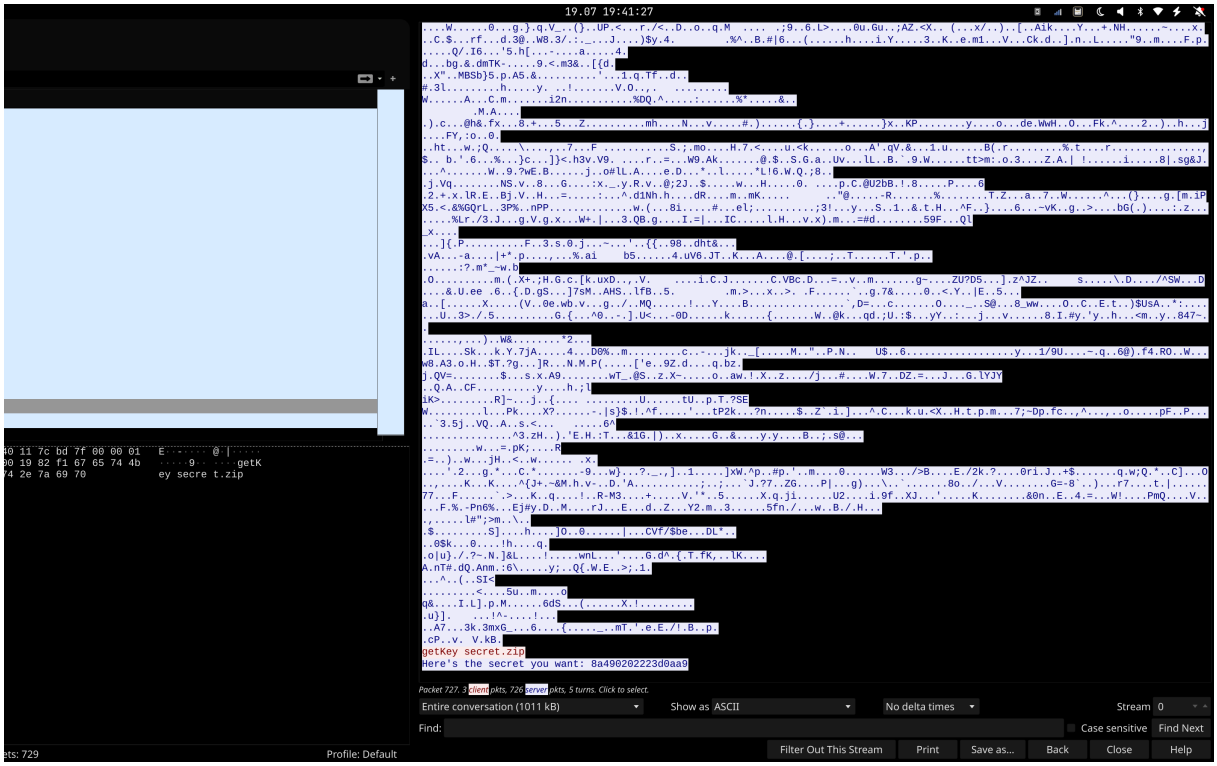
Saved it all to a file and started looking on what was going on.

The first thing that I noticed was that a `player` was logging on and did a `GET` to a `secret.zip` that was what we were trying to also get.

After that the server sent an `INFO 723 999734` message and then the ZIP file.

Later I found that those numbers were the number of `UDP packets` that the server was sending with the ZIP, `723`, and the total size of the ZIP, `999734`.

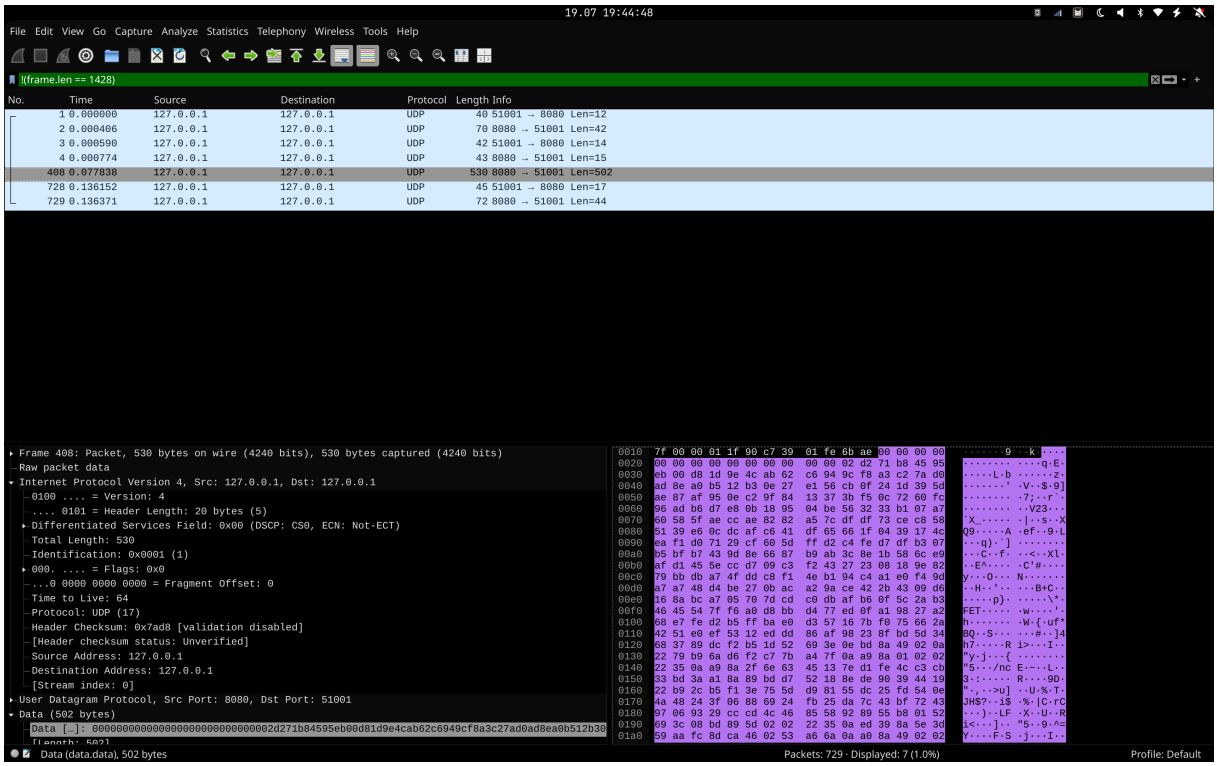
At the end of the stream I noticed 2 packets, one with the instruction to get a `Key` and then the key `8a490202223d0aa9`.



I knew that the UDP streams would come unordered and that the order of the streams needed to be present in the payload somehow.

I then started to go through the packets one by one and looking at what they had different.

I noticed, after a lot of time, that on the `32th nibble` of the payload of each UDP packet on the first `9` packets there was what could possibly be the order of the packet. I took notice of that and searched a little bit more. After a while I used the filter `!(frame.len == 1428)` to remove (what I thougt were all the ZIP packets) to see if I miss something and I found a smaller message with the length of 530 bytes.



I then started to write a python script to order the packets.

I then went through the logic and thought about using the key as a XOR key for the packets to decrypt it.

I ended up with the following script to order and decode the secret zip (took me like some time and half of that time was to understand that the len of the packet needed to be something like `>= 400` and not `>= 530`):

```
from scapy.all import *
scapy_cap = rdpcap('capture.pcap')
key = bytes.fromhex('8a490202223d0aa9')
ordered_pkts = []
for packet in scapy_cap:
  if len(packet[Raw]) >= 400:
    data = bytes(packet[Raw])
    # first 16 bytes are the order of the packet
    order = data[:16]
    # they are in big endian
    order_int = int.from_bytes(order, byteorder='big')
    ordered_pkts.append((order_int, packet))
    ordered_pkts.sort()
zip_file = b''
for packet in ordered_pkts[0:]:
  data = bytes(packet[1][Raw])
  payload = data[16:]
  # need to xor the payload with the key 8a490202223d0aa9
  decrypted_payload = b''
  for i in range(len(payload)):
    decrypted_payload += bytes([payload[i] ^ key[i % len(key)]])
  zip_file += decrypted_payload
with open('archive.zip', 'wb') as f:
  f.write(zip_file)
```

That got me `archive.zip` and inside the files `lorem.txt` and `flag.txt` with the flag.